CHAPTER 3 · TRANSPORT LAYER

# 3.4 Reliable Data Transfer

## Principles of Reliable Data Transfer

Kurose & Ross, 8th Ed.

INTRO

# The Fundamental Challenge

Section 3.4: Introduction

**How can two entities communicate RELIABLY over an UNRELIABLE channel?**

### What the unreliable channel can do

· Corrupt bits in a packet (noise on physical links)
· Lose entire packets (router buffer overflow)
· Deliver packets out of order

IP does all three of these → it is UNRELIABLE (best-effort).

Yet TCP delivers data in perfect order, without errors, and without missing bytes.

### Our approach: incremental protocol design

We build protocols rdt1.0 → rdt2.0 → rdt2.1 → rdt2.2 → rdt3.0, each addressing a harder channel model.

Then we show rdt3.0 is too slow → motivation for pipelining, GBN, and Selective Repeat.

Scope: UNIDIRECTIONAL data transfer (sender → receiver).

**Protocol interfaces:**

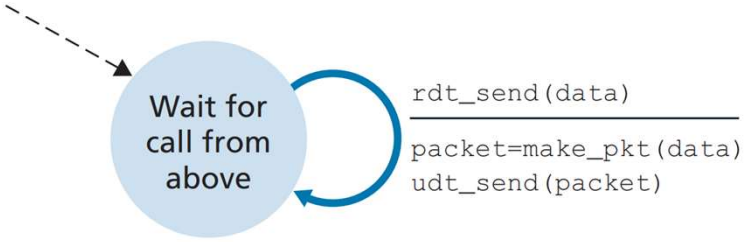
- `rdt_send(data)` → application calls to send
- `rdt_rcv(pkt)` → called on packet arrival
- `udt_send(pkt)` → send over UNRELIABLE channel
- `deliver_data(data)` → deliver to application above

# rdt1.0 — Reliable Transfer over a Perfect Channel

Section 3.4.1: Building rdt

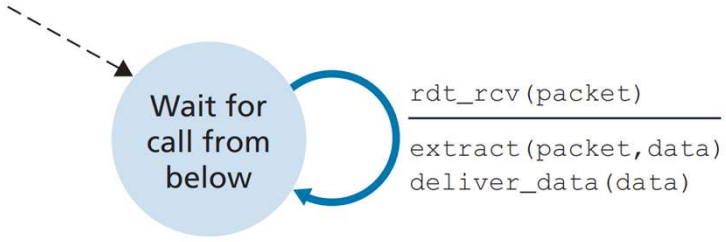
**Simplest case: the underlying channel is PERFECT — no bit errors, no packet loss**

**Sender FSM:**




*(self-loop — one state only)*

**Receiver FSM:**




*(self-loop — one state only)*

**No feedback needed — nothing can go wrong! No ACKs, no timers, no sequence numbers.**

*rdt1.0 is our baseline — every additional complexity in later protocols is forced by the unreliable channel.*

rdt2.0

# rdt2.0 — Channel with Bit Errors: ARQ Protocols

Section 3.4.1: Building rdt

**New channel model: bits in a packet can be CORRUPTED. All packets are delivered, but some arrive with errors.**

Analogy: dictating a message over the phone. Receiver says "OK" after each sentence (ACK) or "Please repeat that" (NAK).

**ARQ = Automatic Repeat reQuest. Three new mechanisms required:**

### 1 Error Detection

Checksum added to data packet. Receiver can detect if any bits were flipped in transit.

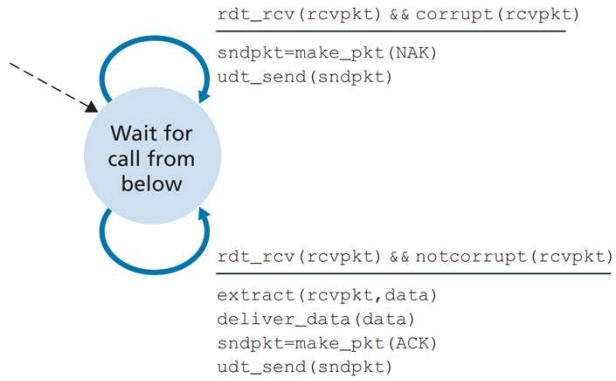
### 2 Receiver Feedback

ACK (positive): packet received correctly.
NAK (negative): packet received with errors.
Sent back from receiver to sender.

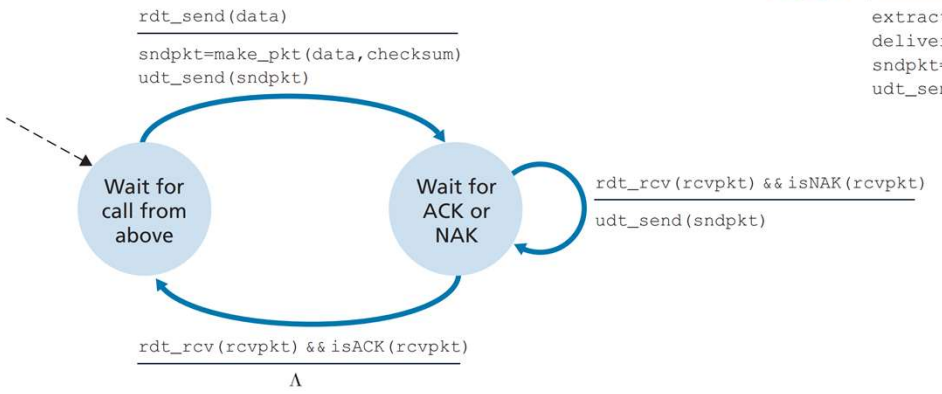
### 3 Retransmission

If sender receives a NAK: retransmit the packet.
If ACK: proceed to next packet.
Stop-and-wait: one packet at a time.

**Receiver FSM:**




**Sender FSM:**



# rdt2.0 — The Fatal Flaw

Section 3.4.1: Building rdt

**Fatal Flaw: What happens if the ACK or NAK itself gets CORRUPTED?**

### The Problem

Sender receives a garbled ACK/NAK.

It cannot tell whether the receiver:
(a) got the packet correctly (ACK)
(b) got it with errors (NAK)

What should the sender do?

### Attempted Solutions

Option 1: Ask receiver to repeat its response.
→ But what if THAT is also corrupted? Infinite regress.

Option 2: Just retransmit.
→ Works, BUT the receiver may get a DUPLICATE and cannot tell if it is a new packet or a retransmission!

### The Fix → rdt2.1: Add Sequence Numbers

Add a SEQUENCE NUMBER field to each data packet.

The receiver uses the sequence number to determine:
· If the incoming packet is NEW (different seq #)
· If it is a DUPLICATE retransmission (same seq #)

For a stop-and-wait protocol: a 1-BIT sequence number is sufficient — alternating 0 and 1.

**rdt2.0 is broken — it cannot handle corrupted acknowledgments. Sequence numbers fix this.**

# rdt2.1 — Sequence Numbers · rdt2.2 — NAK-Free

Section 3.4.1: Building rdt

## rdt2.1 — Adds Sequence Numbers

- 1-bit sequence number: packets labeled 0 or 1 alternately.
- Sender: 4 states — wait for call 0 / send pkt 0 / wait for ACK-NAK 0 / ... (mirror for seq 1).
- Receiver: 2 states — wait for pkt 0 / wait for pkt 1.
- On receiving pkt with WRONG seq #: discard it (duplicate!) but still send ACK.
- On receiving CORRECT seq #: deliver to app, send ACK, advance expected seq #.
- Handles corrupted ACK/NAK: sender retransmits; receiver detects duplicate via seq #.

## rdt2.2 — NAK-Free Protocol

- Key insight: NAK can be eliminated entirely.
- Instead of NAK, receiver sends ACK for the LAST CORRECTLY RECEIVED packet.
- If sender gets TWO ACKs for the same packet (duplicate ACKs): infers the next packet was NOT received correctly → retransmit.
- ACK must now include the sequence number of the packet being acknowledged.
- This is exactly TCP's mechanism: duplicate ACKs trigger retransmission.
- One message type (ACK) instead of two (ACK + NAK) — simpler protocol.

# rdt3.0 — Lossy Channel with Bit Errors: Adding a Timer

Section 3.4.1: Building rdt

**New challenge: the channel can now LOSE PACKETS ENTIRELY - the sender may wait forever for an ACK**

**rdt3.0 = rdt2.2 + TIMER | Also called: the Alternating-Bit Protocol (seq numbers 0 and 1 alternate)**

### The Solution: Timer + Retransmission

Sender starts a COUNTDOWN TIMER every time it sends a packet.

If no ACK received within the timeout period:
→ Assume packet (or its ACK) was lost
→ RETRANSMIT the packet
→ Restart the timer

If ACK arrives before timeout:
→ Stop the timer
→ Proceed to next packet.

### The Key Insight

The sender CANNOT distinguish between:
Case 1: the data packet was lost
Case 2: the ACK was lost
Case 3: the packet/ACK was just delayed

In ALL three cases, the action is IDENTICAL:
→ RETRANSMIT

Cases 2 & 3 produce DUPLICATE packets — but rdt2.2's sequence numbers already handle duplicates correctly.

| Checksum | Seq Numbers | ACK | Timer |
|---|---|---|---|
| Detect bit errors | Handle duplicates | Receiver feedback | Handle packet loss |

**These four mechanisms — checksum, sequence numbers, ACKs, and timers — are the building blocks of ALL reliable data transfer protocols, including TCP.**

rdt3.0

# rdt3.0 — Four Operating Scenarios

Section 3.4.1: Building rdt

### Scenario A — No Loss

Sender: pkt0 → timer starts → ACK0 arrives → stop timer
→ pkt1 → ACK1 → ...
Normal operation. No retransmissions needed.

### Scenario B — Lost Data Packet

Sender sends pkt1 → pkt1 disappears in network
→ No ACK arrives → TIMEOUT fires
→ Sender retransmits pkt1 → ACK1 received → proceed.
The data packet never reached the receiver.

### Scenario C — Lost ACK

Sender sends pkt0 → Receiver sends ACK0 → ACK0 LOST in network
→ Sender times out → retransmits pkt0
→ Receiver: duplicate pkt0 detected (already saw seq 0)
→ Discards data, resends ACK0 → sender moves on.

### Scenario D — Premature Timeout

ACK0 is delayed (not lost) → timer fires before ACK0 arrives
→ Sender retransmits pkt0 → receiver sends ACK0 again (duplicate)
→ Original ACK0 eventually arrives — sender ignores it
→ Duplicate ACK0 from retransmission also arrives — ignored.
No harm done: sequence numbers handle all duplicates.

**Key: the sender cannot distinguish cases B, C, D — it always retransmits on timeout. Sequence numbers handle the duplicates.**

PERFORMANCE

# Performance of Stop-and-Wait: rdt3.0 is Too Slow

Section 3.4.2: Pipelined Protocols

**rdt3.0 is correct — but its performance on real networks is catastrophically poor**

**Setup: West Coast → East Coast · R = 1 Gbps · L = 1,000 bytes (8,000 bits) · RTT = 30 ms**

| | |
|---|---|
| **Transmission time:** | `t_trans = L / R = 8,000 bits / 10⁹ bps = 0.008 ms = 8 µs` |
| **Time until ACK received:** | `RTT + t_trans = 30 ms + 0.008 ms = 30.008 ms` |
| **Sender utilization:** | `U = t_trans / (RTT + t_trans) = 0.008 / 30.008 ≈ 0.00027 = 0.027%` |
| **Effective throughput:** | `Throughput = U × R = 0.00027 × 1 Gbps = 267 kbps (on a 1 Gbps link!)` |

**The sender is IDLE 99.973% of the time. The 1 Gbps link carries only 267 kbps — a factor of 3,700× below capacity!**

PIPELINING

# Pipelining — Filling the Pipe

Section 3.4.2: Pipelined Protocols

### The Idea

Instead of sending ONE packet and waiting for its ACK, send MULTIPLE packets before waiting.

W = window size = number of packets that can be "in flight" (sent but not yet acknowledged) simultaneously.

Utilization formula:

$U = (W \times L/R) / (RTT + L/R)$

### Example: how large must W be?

Using our 1 Gbps / 30ms RTT setup:

For U ≈ 100%:
$W \geq (RTT + L/R) / (L/R) = 30.008 / 0.008 \approx 3{,}751$ packets

With W = 3,751: U ≈ 100%  (full link utilization)

## Pipelining consequences — what must change:

**1 Larger sequence number space**

W in-flight packets need W distinct seq numbers. 1-bit (values 0,1) is no longer enough. TCP uses 32 bits.

**2 Sender buffering**

Sender must buffer all sent-but-unACKed packets — may need to retransmit them on timeout.

**3 Receiver buffering**

Receiver may need to buffer out-of-order arriving packets (depends on protocol: GBN vs SR).

**4 Error recovery approach**

Two options: Go-Back-N (simple receiver, costly on error) or Selective Repeat (smarter).

GBN

# Go-Back-N (GBN) — Overview

Section 3.4.3: Go-Back-N

**GBN: sender can have up to N unacknowledged packets in the pipeline. N = window size.**

**Sender's view of the sequence number space (window size N):**


### Cumulative ACKs

ACK(n) means all packets up through AND INCLUDING n have been correctly received.

### Timeout → Go-Back-N

On timeout: retransmit ALL packets from base to nextseqnum-1. This is where the protocol gets its name.

### Discard out-of-order

Receiver DISCARDS any out-of-order packet. Sends ACK for last correctly received in-order packet.

### Sequence number constraint

With k-bit seq numbers: window size N ≤ 2^k - 1 (one less than full range, to distinguish old ACKs from new).

GBN

# GBN — Sender and Receiver Events

Section 3.4.3: Go-Back-N

## GBN Sender — 3 Events

### 1. Data from above

If window NOT full: make packet, send it, increment *nextseqnum*.
If window IS full: refuse the data (upper layer must retry later).

### 2. ACK(n) received

Cumulative: advance *base = n + 1*.
If more unACKed packets remain in window: restart timer.
If no outstanding packets in window: stop timer.

### 3. Timeout!

GO-BACK-N: retransmit ALL packets in window:
pkt[base], pkt[base+1], ..., pkt[*nextseqnum-1*]
Restart timer.

## GBN Receiver — Extremely Simple

**Packet(n) correct AND n == *expectedseqnum*:**

deliver data to upper layer
send ACK(n)
*expectedseqnum++*

**ANY other packet (wrong seq # or corrupt):**

DISCARD the packet
Resend ACK(*expectedseqnum* − 1)
→ No receiver buffering needed!

**GBN receiver only needs ONE variable: *expectedseqnum*.**
**Simplicity at the cost of discarding correctly received out-of-order packets.**

GBN

# GBN in Operation — Example (Window N = 4)

Section 3.4.3: Go-Back-N

| Sender | Receiver |
| --- | --- |
| send pkt0 | receive pkt0 → deliver → ACK0 |
| send pkt1 | receive pkt1 → deliver → ACK1 |
| send pkt2 ← LOST | (pkt2 never arrives) |
| send pkt3 | receive pkt3 → OUT OF ORDER → DISCARD → ACK1 |
| receive ACK0 → send pkt4 | receive pkt4 → discard → ACK1 |
| receive ACK1 → send pkt5 | receive pkt5 → discard → ACK1 |
| ⏱ **pkt2 TIMEOUT!** | |
| retransmit pkt2 | receive pkt2 → deliver → ACK2 |
| retransmit pkt3 | receive pkt3 → deliver → ACK3 |
| retransmit pkt4 | receive pkt4 → deliver → ACK4 |
| retransmit pkt5 | receive pkt5 → deliver → ACK5 |

SR

# Selective Repeat (SR) — Overview

Section 3.4.4: Selective Repeat

**GBN problem: on timeout, it retransmits ALL W packets — even those already correctly received.**

**SR solution: retransmit ONLY the packets that are lost or corrupted — hence "selective" repeat**

### SR vs GBN — Key Differences

1. RECEIVER has its own window of size N.
   Receiver BUFFERS out-of-order packets.

2. Per-packet timers at the sender.
   Only the lost packet is retransmitted.

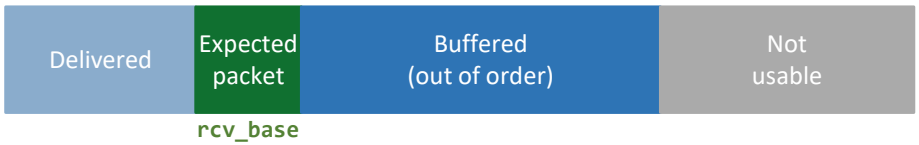
3. INDIVIDUAL ACKs: ACK(n) means specifically packet n was received correctly — not cumulative.

4. Window constraint: N ≤ (seq space) / 2

### Sender and Receiver both have windows:

**Sender window:**


**Receiver window:**




*Note: sender and receiver windows do NOT always coincide.*

**SR avoids unnecessary retransmissions — critical when window is large and errors are frequent**

SR

# SR — Sender and Receiver Events

Section 3.4.4: Selective Repeat

## SR Sender — 3 Cases

### 1. Data from above

If within sender window: packetize & send, start per-packet timer.

### 2. Timeout for packet n

Retransmit ONLY packet n.
Restart ONLY timer for packet n.
→ NOT the whole window!

### 3. ACK(n) received

Mark packet n as received.
If n == send_base: advance window to next unACKed packet.

## SR Receiver — 3 Cases

### 1. Packet in [rcv_base, rcv_base+N-1]

→ Send individual ACK(n).
If n ≠ rcv_base: buffer the packet.
If n == rcv_base: deliver n + all consecutive buffered packets to upper layer. Advance rcv_base.

### 2. Packet in [rcv_base-N, rcv_base-1]

→ Send ACK(n) ANYWAY — even though this is a previously received packet.
The sender may still need this ACK to advance its window.

### 3. Otherwise

→ Ignore the packet.
It is outside both the current and previous window.

SR

# SR in Operation — Example (Window N = 4)

Section 3.4.4: Selective Repeat

| Sender | Receiver |
|---|---|
| send pkt0 | rcv pkt0 → deliver → ACK0 |
| send pkt1 | rcv pkt1 → deliver → ACK1 |
| send pkt2 ← LOST | (pkt2 never arrives) |
| send pkt3 | rcv pkt3 → BUFFER → ACK3 |
| receive ACK0 → send pkt4 | rcv pkt4 → BUFFER → ACK4 |
| receive ACK1 → send pkt5 | rcv pkt5 → BUFFER → ACK5 |
| receive ACK3, ACK4, ACK5 | |
| ⏱ **pkt2 TIMEOUT!** | |
| **retransmit pkt2 ONLY** | rcv pkt2 → deliver pkt2+3+4+5 → ACK2 |
| receive ACK2 → window moves | Window moves forward ✓ |

**SR retransmits ONLY pkt2 — not pkt3, pkt4, pkt5. Batch delivery when rcv_base is filled.**

SR

# SR Window Size Constraint — Why W ≤ (Seq Space) / 2?

Section 3.4.4: Selective Repeat

**SR requires: window size ≤ sequence number space / 2**

**Illustration: 2-bit seq numbers (values: 0, 1, 2, 3), Window = 3**

Seq space = 4. Window = 3 > 4/2 = 2 ← TOO LARGE!

Scenario:
- Sender sends pkt0, pkt1, pkt2
- Receiver gets all three → sends ACK0, ACK1, ACK2
- ALL ACKs are lost in network
- Sender times out → retransmits pkt0
- Receiver's window has advanced to [3, 0, 1]
- pkt0 arrives — is it a NEW pkt0 or RETRANSMITTED pkt0?

→ RECEIVER CANNOT TELL!

May deliver a DUPLICATE as if it were new data.

**Correct: 2-bit seq numbers, Window = 2**

Window = 2 ≤ 4/2 = 2 ← OK

Same scenario:
- Sender sends pkt0, pkt1
- Receiver window: [0, 1] → receives → ACKs lost
- Sender retransmits pkt0
- Receiver's window has advanced to [2, 3]
- pkt0 arrives — seq 0 is NOT in [2, 3]

→ Receiver recognizes it as OLD → reACKs it
→ NO CONFUSION!

Summary:
GBN: window ≤ 2^k − 1
SR: window ≤ 2^k / 2 (half the space)

**Violating the SR window constraint causes the receiver to mistake retransmissions for new data — silent data corruption.**

GBN vs SR

# GBN vs. Selective Repeat — Full Comparison

Section 3.4.3–3.4.4

| Dimension | Go-Back-N (GBN) | Selective Repeat (SR) |
|---|---|---|
| **Window type** | Sender only (receiver = 1 expected seq #) | Sender AND receiver (both size N) |
| **ACK type** | Cumulative: ACK(n) = all ≤ n received | Individual: ACK(n) = packet n received |
| **On timeout** | Retransmit ALL packets in window | Retransmit ONLY the lost packet |
| **Receiver buffering** | NONE — out-of-order packets discarded | Up to N−1 out-of-order packets buffered |
| **Receiver complexity** | Simple — only expectedseqnum needed | More complex — window + per-packet state |
| **Window constraint** | N ≤ 2^k − 1 | N ≤ 2^k / 2 (HALF the seq space) |
| **Efficiency on errors** | Wastes BW: retransmits even good packets | Efficient: retransmits only needed pkts |
| **Best suited for** | Low error rates | Higher error rates / large windows |
| **TCP resemblance** | Cumulative ACKs (like GBN) | Buffers out-of-order (like SR) — HYBRID |

*TCP uses cumulative ACKs (GBN-like) but does NOT discard out-of-order segments (SR-like) — a practical hybrid.*

SUMMARY

# Summary of rdt Mechanisms (Table 3.1)

Section 3.4: Summary

**These four building blocks underpin ALL reliable data transfer — at any layer**

| | |
|---|---|
| **Checksum** | Detect bit errors in a transmitted packet. Every rdt protocol from rdt2.0 onward. |
| **Timer** | Timeout/retransmit a packet that may be lost. Timeouts can cause duplicates (handled by seq#). From rdt3.0. |
| **Seq Numbers** | Sequential numbering for packets. Gaps detect lost packets. Duplicates detect retransmissions. From rdt2.1. |
| **ACK (positive)** | Receiver tells sender which packets arrived correctly. Can be individual (SR) or cumulative (GBN/TCP). From rdt2.0. |
| **NAK (negative)** | Receiver tells sender which packet was NOT received correctly. Eliminated in rdt2.2 via duplicate ACKs. |
| **Window/Pipeline** | Sender restricted to a sequence number range. Multiple in-flight packets → higher utilization. GBN & SR. |

*These mechanisms are not specific to the transport layer — they can be implemented at any protocol layer.*

# Summary

3.4 — Principles of Reliable Data Transfer

- rdt1.0: trivial — perfect channel. No mechanisms needed.
- rdt2.0: bit errors → ARQ: checksum + ACK/NAK + retransmission. Fatal flaw: corrupted ACK/NAK.
- rdt2.1: adds 1-bit sequence numbers (0/1). Detects and discards duplicates.
- rdt2.2: eliminates NAK. Duplicate ACKs serve as implicit NAK. TCP uses this.
- rdt3.0: adds TIMER to handle packet loss. All four mechanisms: checksum, seq#, ACK, timer.
- Stop-and-wait utilization: U = (L/R) / (RTT + L/R). On 1 Gbps / 30ms RTT = only 0.027%.
- Pipelining: send W packets in flight. U = (W × L/R) / (RTT + L/R). Need W ≈ 3,750 for full use.
- GBN: window N, cumulative ACKs, timeout → retransmit ALL. Receiver discards out-of-order.
- SR: window N each side, individual ACKs, timeout → retransmit ONLY lost packet. Window ≤ 2^k/2.
- Four fundamental mechanisms underpin ALL reliable protocols: checksum · seq# · ACK · timer.